



5

25

3750 sp

نظري

كلية الهندسة المعلوماتية

السنة الرابعة-النظم والشبكات

## خدمات طبقة النقل

د. علي حماد



RB Informatics; 29/04/2026

بروتوكولات الإتصال

## السلام عليكم و رحمة الله و بركاته

### المقدمة

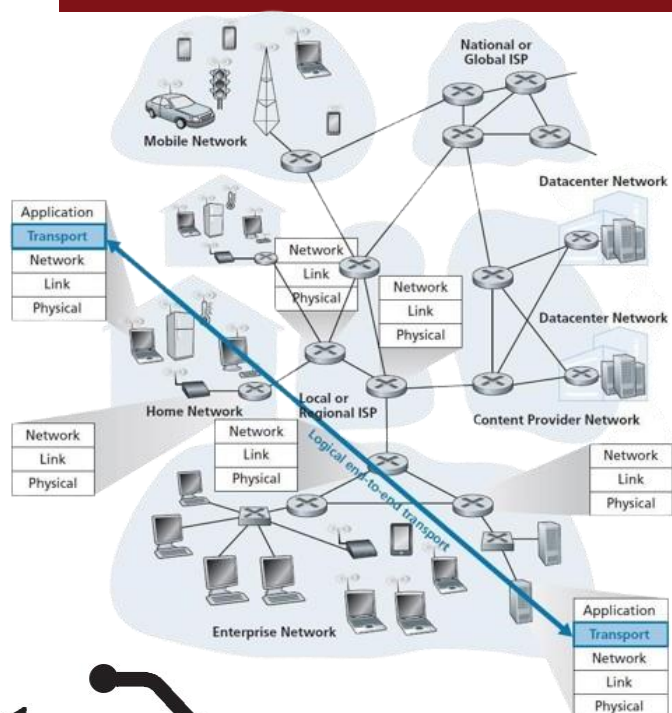
عند التحدث مع صديق لي على تطبيق واتس اب تكون المحادثة عبارة عن العمليات العديدة التي يجريها التطبيق في جهازي و جهاز صديقي.

إذا أين توجد هذه العمليات؟ في طبقة التطبيقات (Application Layer).

ومن المسؤول عن تواصل هذه العمليات أو بمعنى آخر من المسؤول عن توصيل البيانات بين المرسل والمستقبل؟ بالطبع هي طبقة النقل (Transport Layer).

وخلف هذا الاتصال الذي تؤمنه طبقة النقل الكثير من التفاصيل والإجراءات التي حققتها والتي سنتعرف عليها معاً في هذه المحاضرة.

### خدمات طبقة النقل (Introduction and Transport-Layer Services)



يوفر بروتوكول طبقة النقل اتصالاً منطقياً (logical communication) بين عمليات التطبيقات (processes) التي تعمل على أجهزة مستضيفة (hosts) مختلفة. حيث تستخدم عمليات التطبيقات الاتصال المنطقي الذي توفره طبقة النقل لإرسال الرسائل لبعضها البعض دون القلق بشأن تفاصيل البنية التحتية المادية (physical infrastructure) المستخدمة لنقل هذه الرسائل.

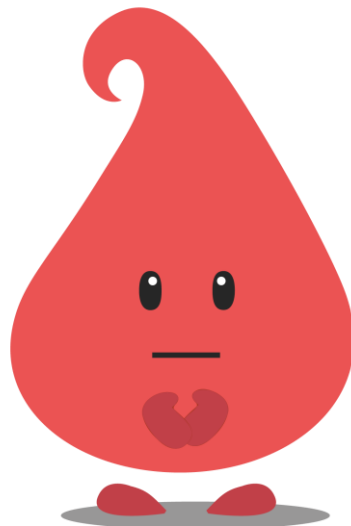
## تفاصيل عملية الإرسال والاستقبال

عندما تريد إرسال ملف كبير أو رسالة عبر الإنترنت تتبع طبقة النقل الخطوات التالية:

1. تقوم طبقة النقل بتحويل رسائل طبقة التطبيقات إلى حزم خاصة تُعرف باسم «أجزاء طبقة النقل» (transport-layer segments)، وذلك عن طريق تقسيم رسائل التطبيق إلى أجزاء أصغر وإضافة ترويسة طبقة النقل (transport-layer header) إلى كل جزء.
2. تقوم طبقة النقل بتمرير هذه القطعة (segment) إلى طبقة الشبكة (network layer) في الجهاز الطرفي المرسل، حيث يتم تغليفها (encapsulated) داخل حزمة طبقة الشبكة (datagram) ومن ثم إرسالها إلى الوجهة.
3. في جانب الاستقبال (receiving side): تقوم طبقة الشبكة باستخراج قطعة طبقة النقل من حزمة البيانات (datagram) ثم تمررها إلى طبقة النقل التي تعالج القطعة المستلمة وتجعل البيانات متاحة للتطبيق (المستقبل).

### ملاحظة:

- يتم تنفيذ بروتوكولات طبقة النقل في الأجهزة الطرفية (end systems) وليس في موجهات الشبكة (routers)، حيث أن الراوتر فقط توجيه الحزم بناءً على IP.
  - كما أن موجهات الشبكة (network routers) تتعامل فقط مع حقول طبقة الشبكة الخاصة بحزمة البيانات (datagram) أي أنها لا تفحص حقول قطعة طبقة النقل transport-layer segment المغلفة داخل حزمة البيانات.
- بمعنى:** أن الراوتر يتصرف مثل ساعي البريد تماماً يقرأ فقط العنوان الخارجي المكتوب على الطرد (وهو عنوان IP الموجود في طبقة الشبكة) ليعرف إلى أين يوجهه و لا يفتح الطرد ليرى ما بداخله (لا يفحص حقول طبقة النقل أو ال Segment).



## العلاقة بين طبقتي النقل والشبكة

### (Relationship Between Transport and Network Layers)

- تتواجد بروتوكولات طبقة النقل في الأجهزة الطرفية (end systems) فقط. داخل الجهاز الطرفي يقوم بروتوكول النقل بنقل الرسائل من عمليات التطبيق إلى حافة الشبكة (network edge) والعكس صحيح، ولكن ليس له أي تحكم في كيفية نقل هذه الرسائل داخل نواة الشبكة (network core).
- (المقصود بـ نواة الشبكة أي شبكة الإنترنت العالمية المكونة من آلاف الموجهات (Routers) والروابط)
- غالباً ما تكون الخدمات التي يمكن لبروتوكول النقل تقديمها مقيدة بنموذج الخدمة (service model) لبروتوكول طبقة الشبكة الأساسي: حيث إذا كان بروتوكول طبقة الشبكة لا يمكنه توفير ضمانات للتأخير (delay) أو عرض النطاق الترددي (bandwidth) لقطع طبقة النقل المرسله بين الأجهزة، فإن بروتوكول طبقة النقل لا يمكنه بدوره توفير ضمانات للتأخير أو عرض النطاق الترددي لرسائل التطبيق المرسله بين العمليات.
- و لكن يوجد طبعاً استثناءات أي :**
- يمكن أحيانا تقديم خدمات معينة بواسطة بروتوكول النقل حتى عندما لا يقدم بروتوكول الشبكة الأساسي الخدمة المقابلة لها في طبقة الشبكة.
- على سبيل المثال، يمكن لبروتوكول النقل تقديم خدمة نقل بيانات موثوقة (reliable data transfer service) للتطبيق حتى عندما يكون بروتوكول الشبكة الأساسي غير موثوق (unreliable).
- و أيضاً يمكن لبروتوكول النقل استخدام التشفير (encryption) لضمان عدم قراءة رسائل التطبيق من قبل المتطفلين، حتى عندما لا تتمكن طبقة الشبكة من ضمان سرية أجزاء طبقة النقل (transport-layer segments).

## طبقة النقل في الإنترنت

### (Overview of the Transport Layer in the Internet)

يوفر الإنترنت بروتوكولين متميزين في طبقة النقل لتستفيد منهما طبقة التطبيقات:

1. **UDP (User Datagram Protocol)** - بروتوكول حزم بيانات المستخدم: يقدم خدمة غير موثوقة وغير قائمة على اتصال (unreliable, connectionless service) للتطبيق (المستدعي).
2. **TCP (Transmission Control Protocol)** - بروتوكول التحكم في النقل: يقدم خدمة موثوقة وقائمة على اتصال (reliable, connection-oriented service) للتطبيق (المستدعي).

## كيف تعمل طبقة الشبكة (التي تقع تحت TCP و UDP مباشرة) عبر بروتوكول IP:

الإنترنت بطبيعته الفيزيائية مبني على بروتوكول IP وهو بروتوكول يتصرف مثل ساعي بريد «يبدل جهده» للإيصال الرسالة (segments) لكنه لا يضمن أي شيء:

1. لا يضمن وصول الحزمة (قد تضيع بسبب ازدحام الراوترات).
2. لا يضمن الترتيب (قد تصل الحزمة رقم 5 قبل الحزمة رقم 3 نتيجة اتخاذ مسارات مختلفة).
3. لا يضمن سلامة المحتوى (قد تتلف البيانات أو تتغير في الطريق بسبب تداخل الإشارات).

ولهذه الأسباب يُقال إن بروتوكول IP هو خدمة غير موثوقة (unreliable service).

### فائدة أريسيةزية:

بروتوكول TCP يعمل فوق بروتوكول IP غير الموثوق. ورغم أن الـ IP يضيع الحزم ويبعثرها إلا أن الـ TCP (في طبقة النقل بالأجهزة الطرفية) يمتلك آليات ذكية لإعادة الترتيب وطلب إعادة الإرسال ليوفر للمستخدم في النهاية اتصالاً موثقاً وأمناً بنسبة 100٪.

## خدمات بروتوكولي TCP مقابل UDP

### التوصيل من عملية إلى عملية

بروتوكول الإنترنت (IP) وظيفته فقط هي توصيل البيانات من جهاز كمبيوتر إلى جهاز كمبيوتر آخر (Host-to-Host) هو يكفي بإيصال الرسالة إلى عنوان الـ IP الخاص بالجهاز المستهدف وتنتهي مهمته هناك. وعندما تصل البيانات إلى جهازك يوجد داخل نظام التشغيل عشرات البرامج والعمليات (Processes) التي تعمل في نفس الوقت (متصفح، لعبة، بريد إلكتروني...) و هنا يأتي دور طبقة النقل (عبر TCP أو UDP) لتأخذ الرسالة من منفذ الجهاز وتحدد أي برنامج دقيق داخل الجهاز يجب أن يستلمها. و يُسمى هذا التحول والتوزيع بـ **التجميع وفك التجميع (Multiplexing and Demultiplexing)** وهو يعتمد على ما يُعرف بـ أرقام المنافذ (Port Numbers).

### فحص سلامة البيانات

يوفر بروتوكولا UDP و TCP أيضاً فحصاً لسلامة البيانات (integrity checking) عن طريق تضمين حقول للكشف عن الأخطاء (error-detection fields) في ترويسات القطع (segments' headers) الخاصة بهما. هاتان الخدمتان (التوصيل من عملية إلى عملية وفحص الأخطاء) هما الخدمتان الوحيدتان اللتان يقدمهما بروتوكول UDP.

لذلك يمكن القول أن UDP مثل بروتوكول IP هو خدمة غير موثوقة (unreliable service) - لا يضمن أن البيانات المرسله ستصل سليمة أو تصل على الإطلاق إلى العملية الوجهة.

## خدمات بروتوكول TCP وآلية عمله

يوفر بروتوكول TCP نقلاً موثقاً للبيانات (reliable data transfer) من خلال استخدام:

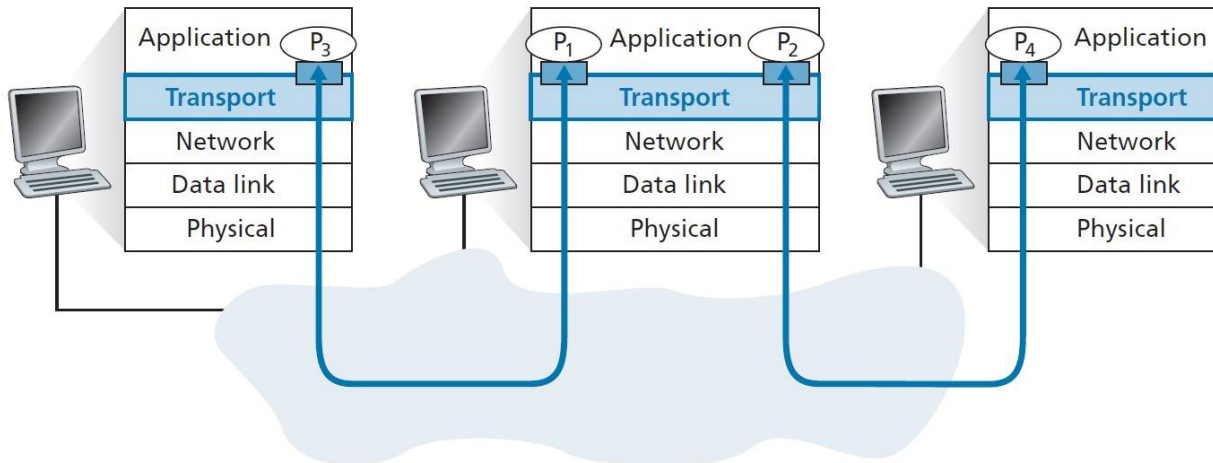
1. التحكم في التدفق (flow control)
2. الأرقام التسلسلية (sequence numbers)
3. الإشعارات بالاستلام (acknowledgments)
4. المؤقتات (timers)

وبذلك يضمن بروتوكول TCP تسليم البيانات من العملية المرسله إلى العملية المستقبلة بشكل صحيح وبالترتيب. يوفر بروتوكول TCP أيضاً التحكم في الازدحام (congestion control).

ولا يُعد التحكم في الازدحام خدمة مقدمة للتطبيق المستدعي بقدر ما هو خدمة للإنترنت ككل أي هي خدمة تحقق المصلحة العامة حيث : التحكم في الازدحام **يمنع** ارسال كمية مفرطة من البيانات و حدوث ازدحام (traffic).

يسعى بروتوكول TCP جاهداً لمنح كل اتصال يمر عبر رابط مزدحم حصة متساوية (equal share) من النطاق الترددي للرابط (link bandwidth) ويتم ذلك عن طريق تنظيم المعدل (regulating the rate) الذي ترسل به البيانات .

## التجميع وفك التجميع (Multiplexing and Demultiplexing)



Key:

○ Process    ■ Socket

- عند الجهاز المضيف المستقبل (destination host) تحصل طبقة النقل على الأجزاء (segments) من طبقة الشبكة الموجودة أسفلها مباشرة. وتتحمل طبقة النقل مسؤولية توصيل البيانات الموجودة في هذه الأجزاء إلى عملية التطبيق المناسبة (appropriate application process) التي تعمل على الجهاز المضيف.
- لا تقوم طبقة النقل في الجهاز المضيف المستقبل بتوصيل البيانات مباشرة إلى العملية بل تقوم بتوصيلها إلى مقبس وسيط (intermediary socket).

## فك التجميع (Demultiplexing) عند المستقبل

تُسمى مهمة توصيل البيانات الموجودة في قطعة طبقة النقل (transport-layer segment) إلى المقبس الصحيح (correct socket) باسم **فك التجميع (demultiplexing)**.

## التجميع (Multiplexing) عند المرسل

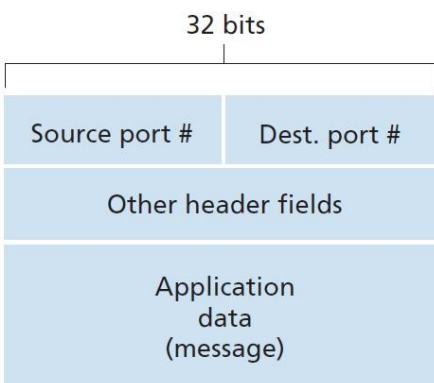
أما مهمة جمع كتل البيانات عند جهاز المضيف (المرسل) من مقابس مختلفة وتغليف كل كتلة بيانات بمعلومات الترويسة (header information) - والتي ستُستخدم لاحقاً في عملية فك التجميع - لإنشاء القطع (segments) ثم تمرير هذه القطع إلى طبقة الشبكة، فتُسمى **التجميع (multiplexing)**.

### فائدة أريسيةزية:

**المقبس (Socket)** هو الواجهة بين طبقة النقل و طبقة التطبيقات و يمكن القول أنه صندوق البريد الافتراضي الخاص بكل برنامج داخل نظام التشغيل حيث أن :  
المتصفح له مقبس خاص به. و تطبيق ديسكورد له مقبس خاص به. و لعبة الأونلاين لها مقبس خاص بها.

يتطلب التجميع و فك التجميع في طبقة النقل ما يلي :

- (1) أن تمتلك المقابس (sockets) معرفات فريدة (unique identifiers).
- (2) أن تحتوي كل قطعة (segment) على حقول خاصة تشير إلى المقبس الذي يجب تسليم هذه القطعة إليه: حقول رقم منفذ المصدر (source port number field) وحقول رقم منفذ الوجهة (destination port number field).

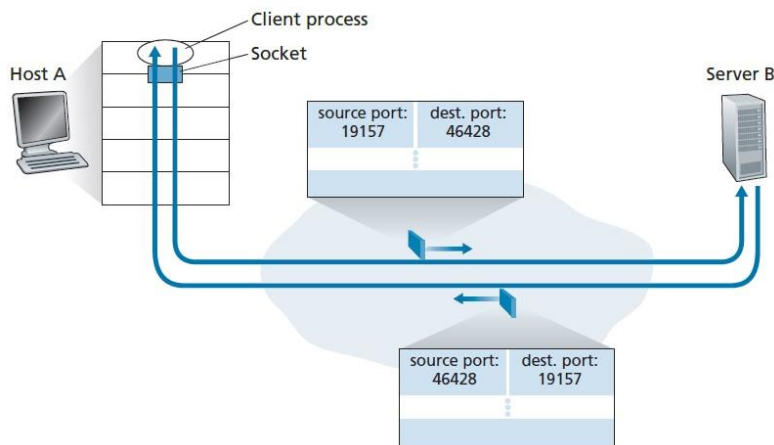


مكونات الرسم التوضيحي للهيكل (32 بت):  
 Source port #: رقم منفذ المصدر.  
 Dest. port #: رقم منفذ الوجهة.  
 Other header fields: حقول الترويسة الأخرى.  
 Application data (message): بيانات طبقة التطبيقات (الرسالة).

- كل رقم منفذ (port number) هو عبارة عن رقم بحجم 16 بت (16-bit number) ويتراوح نطاقه من 0 إلى 65535.
- تُسمى أرقام المنافذ التي تتراوح من 0 إلى 1023 باسم المنافذ المعروفة (well-known port numbers) وهي أرقام مقيدة، مما يعني أنها محجوزة للاستخدام بواسطة بروتوكولات طبقة التطبيقات الشهيرة مثل بروتوكول HTTP (الذي يستخدم رقم المنفذ 80) وبروتوكول FTP (الذي يستخدم رقم المنفذ 21).
- يمكن تعيين رقم منفذ لكل مقبس (socket) في الجهاز المضيف وعندما تصل قطعة (segment) إلى الجهاز المضيف تفحص طبقة النقل رقم منفذ الوجهة (destination port number) الموجود في القطعة وتوجه القطعة مباشرة إلى المقبس المقابل لها

## التجميع وفك التجميع غير القائم على اتصال في بروتوكول UDP (Connectionless Multiplexing and Demultiplexing)

- في بروتوكول UDP يتم تحديد المقبس (UDP socket) بشكل كامل بواسطة ثنائية (two-tuple) **تتكون من:** عنوان IP الوجهة (destination IP address) ورقم منفذ الوجهة (destination port number).
- إذا كانت هناك قطعتان من بروتوكول UDP (UDP segments) تمتلكان عناوين IP مصدر مختلفة و/أو أرقام منافذ مصدر مختلفة، ولكن لهما نفس عنوان IP الوجهة ونفس رقم منفذ الوجهة، فسيتم توجيه هاتين القطعتين إلى نفس العملية الوجهة (destination process) عبر نفس المقبس الوجهة (destination socket).
- يعمل رقم منفذ المصدر (source port number) كجزء من عنوان العودة (return address).
- بمعنى : ارسلت لك بيانات و اعطيك عنواني لترسل الرد لي .



مكونات الرسم التوضيحي (اتصال بين جهاز خادم وعميل):

- Client process / Socket : عملية العميل / المقبس الخاص به في الجهاز (Host A).
- Server B : الجهاز الخادم الوجهة.
- الرسالة الذاهبة للخادم:
  - Source port: 19157 : منفذ المصدر
  - Dest. port: 46428 : منفذ الوجهة
- الرسالة العائدة للعميل:
  - Source port: 46428 : منفذ المصدر
  - Dest. port: 19157 : منفذ الوجهة

## التجميع وفك التجميع القائم على اتصال في بروتوكول TCP (Connection-Oriented Multiplexing and Demultiplexing)

- أحد الفروقات الدقيقة بين مقبس TCP ومقبس UDP هو أن مقبس TCP يتم تحديده بواسطة رباعية (four-tuple) **تكون من**: عنوان IP المصدر، رقم منفذ المصدر، عنوان IP الوجهة، ورقم منفذ الوجهة.
- وبناءً على ذلك عندما تصل قطعة (TCP segment) من الشبكة إلى جهاز مضيف يستخدم الجهاز القيم الأربعة جميعها لتوجيه (فك تجميع) القطعة إلى المقبس المناسب (appropriate socket).
- وبالعكس بروتوكول UDP فإن قطعتي TCP قادمتين وعناوين IP المصدر الخاصة بهما أو أرقام منافذ المصدر مختلفة سيتم توجيههما إلى مقبسين مختلفين تماماً - **باستثناء قطعة TCP التي تحمل طلب إنشاء الاتصال الأصلي**.

### فائدة أريسيزية

ما المقصود بـ **باستثناء قطعة TCP التي تحمل طلب إنشاء الاتصال الأصلي**؟

- نعلم أن بروتوكول ال TCP ينشئ الاتصال قبل الإرسال حيث يرسل حزمة أولى المخصصة لطلب إنشاء الاتصال فقط وتُسمى حزمة SYN .
- و السيرفر يمتلك مقبساً واحداً عاماً مفتوحاً دائماً (مثل المنفذ 80 للمواقع) يسمى "مقبس الترحيب" أو ال Listening Socket. هذا المقبس لا يهتم بمعرفة IP المصدر أو منفذ المصدر للعميل وظيفته الوحيدة هي استقبال طلبات الاتصال من أي غريب في العالم.
- لذلك عندما تصل حزمة طلب الاتصال الأصلي (الطلب الأول) يكسر السيرفر القاعدة فلا ينظر للرباعية كاملة بل يستقبل الحزمة عبر هذا المقبس العام.

مثال: عندما يستقبل نظام التشغيل الخاص بالجهاز المضيف (الذي يُشغّل عملية الخادم/Server process) قطعة طلب اتصال واردة (incoming connection-request segment) موجهة إلى منفذ الوجهة 12000:

1. يحدد موقع عملية الخادم التي تنتظر قبول الاتصال على رقم المنفذ 12000.
2. بعد ذلك تقوم عملية الخادم بإنشاء مقبس جديد وخاص لهذا المستخدم بالذات (new socket).
3. تسجل طبقة النقل في جهاز الخادم القيم الأربعة التالية الموجودة في قطعة طلب الاتصال:
  - a. رقم منفذ المصدر (source port number) الموجود في القطعة.
  - b. عنوان IP للجهاز المضيف المصدر (source host).
  - c. رقم منفذ الوجهة (destination port number) الموجود في القطعة.
  - d. عنوان IP الخاص بالجهاز الخادم نفسه.
4. يتم تحديد مقبس الاتصال المنشأ حديثاً بواسطة هذه القيم الأربع .

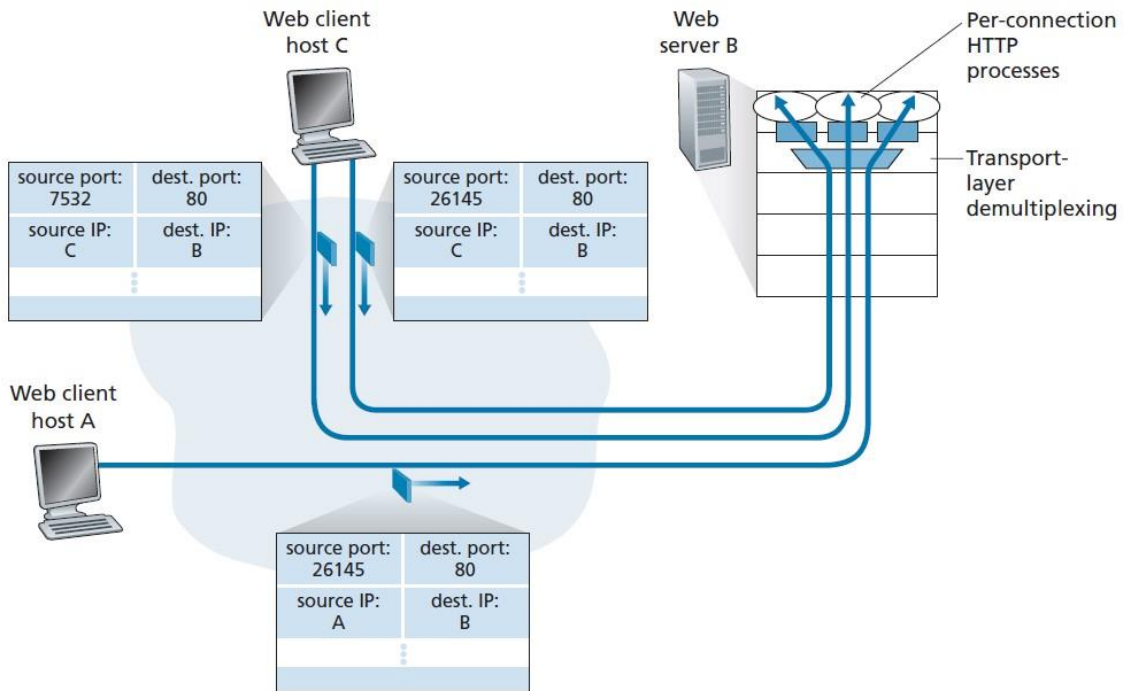
وبالتالي فإن جميع القطع التي تصل لاحقاً والتي يتطابق فيها (منفذ المصدر، وعنوان IP المصدر، ومنفذ الوجهة، وعنوان IP الوجهة) مع هذه القيم الأربع سيتم فك تجميعها وتوجيهها إلى هذا المقبس بالتحديد.

5. ومع إتمام إنشاء اتصال TCP الآن يمكن للعميل والخادم البدء في إرسال البيانات إلى بعضهما البعض.

## مثال 2:

- يقوم الجهاز المضيف C ببدء جلستي HTTP مع الخادم B ويقوم الجهاز المضيف A ببدء جلسة HTTP واحدة مع الخادم B.
- و نظراً لأن الجهاز المضيف A يختار أرقام منافذ المصدر (source port numbers) بشكل مستقل عن الجهاز C، فقد يقوم أيضاً بتعيين منفذ مصدر بالرقم 26145 للاتصال الـ HTTP الخاص به (أي كل من A و C عينا نفس رقم المنفذ).
- مع ذلك سينجح الخادم B في فك التجميع بشكل صحيح (correctly demultiplex) للاتصالين اللذين يمتلكان نفس رقم منفذ المصدر نظراً لأن هذين الاتصالين يمتلكان عناوين IP مصدر مختلفة (different source IP addresses).

**النتيجة:** حتى وإن وصل السيرفر طلبان لهما نفس رقم منفذ المصدر فلن تحدث أي مشكلة وذلك بفضل عناوين الـ IP.



**نلاحظ من خلال الرسم:** جهاز العميل (C) الذي يفتح جلستين منفذ المصدر الأول: 7532 ومنفذ المصدر الثاني: 26145، وكلاهما متجهان إلى منفذ الوجهة 80 في السيرفر B .

جهاز العميل (A) الذي يفتح جلسة واحدة منفذ المصدر: 26145 متجهاً إلى منفذ الوجهة 80 في السيرفر B .

الخادم الوجهة (B) الذي يستقبل الاتصالات الثلاثة عبر طبقة النقل ويوزعها على عمليات الـ HTTP المستقلة

## خوادم الويب وبروتوكول TCP (Web Servers and TCP)

- عندما يرسل العملاء (على سبيل المثال: المتصفحات) قطع بيانات (segments) إلى الخادم فإن جميع هذه القطع ستمتلك منفذ وجهة بالرقم 80 (destination port 80).
- حيث أن كل من قطع إنشاء الاتصال الأولية والقطع التي تحمل رسائل طلبات الـ HTTP ستمتلك منفذ وجهة بالرقم 80.
- لا يوجد دائماً توافق بنسبة واحد لواحد (one-to-one correspondence) بين مقابس الاتصال والعمليات (أي أنه ليس لكل مقبس ستكون له عملية) ففي الواقع غالباً ما تستخدم خوادم الويب عالية الأداء اليوم عملية واحدة فقط (one process) وتقوم بإنشاء خيط فرعي جديد (new thread) مع مقبس اتصال جديد لكل اتصال عميل جديد.
- إذا كان العميل والخادم يستخدمان بروتوكول HTTP المستمر (persistent HTTP) فإنه طوال مدة الاتصال المستمر يتبادل العميل والخادم رسائل الـ HTTP عبر نفس مقبس الخادم (same server socket).
- ولكن إذا كان العميل والخادم يستخدمان بروتوكول HTTP غير المستمر (non-persistent HTTP) فسيتم إنشاء اتصال TCP جديد وإغلاقه مع كل طلب/استجابة، وبالتالي يتم إنشاء مقبس جديد وإغلاقه لاحقاً لكل طلب/استجابة. وهذا الإنشاء والإغلاق المتكرر للمقابس يمكن أن يؤثر بشدة على أداء خادم الويب المزدهم.

## النقل غير القائم على اتصال: بروتوكول UDP (Connectionless Transport: UDP)

- يقوم بروتوكول UDP بأقل قدر ممكن مما يمكن لبروتوكول النقل القيام به. حيث أنه و باستثناء وظيفة التجميع وفك التجميع (multiplexing/demultiplexing) وبعض الفحص الخفيف للأخطاء (light error checking) فإنه لا يضيف شيئاً إلى بروتوكول IP. وفي الواقع إذا اختار مطور التطبيق استخدام UDP بدلاً من TCP فإن التطبيق يتحدث بشكل مباشر تقريباً مع بروتوكول IP.
- يأخذ UDP الرسائل من عملية التطبيق ويرفق حقول رقم منفذ المصدر والوجهة (source and destination port number fields) لخدمة التجميع وفك التجميع، ويضيف حقلين صغيرين آخرين، ثم يمرر القطعة الناتجة (resulting segment) إلى طبقة الشبكة.
- إذا وصلت القطعة إلى الجهاز المضيف المستقبل يستخدم UDP رقم منفذ الوجهة لتسليم بيانات القطعة إلى عملية التطبيق الصحيحة.
- وكما نعلم أنه مع UDP لا توجد عملية مصافحة (no handshaking) بين كيانات طبقة النقل المرسل والمستقبل قبل إرسال القطعة ولهذا السبب يُقال إن بروتوكول UDP غير قائم على اتصال (connectionless).

## لماذا نستخدم بروتوكول UDP؟

### 1 - تحكّم أدق على مستوى التطبيق في نوعية البيانات المرسلّة ووقت إرسالها (Finer application-level control)

حيث أنه بمجرد أن تقوم عملية التطبيق بتمرير البيانات إلى UDP فإنه يقوم فوراً بتغليف البيانات داخل قطعة UDP (UDP segment) وتمرير القطعة على الفور إلى طبقة الشبكة (network layer).  
بينما يمتلك بروتوكول TCP آلية للتحكم في الازدحام (congestion-control mechanism) والتي تقوم بمنع المرسل في طبقة النقل لبروتوكول TCP عندما يصبح رابط واحد أو أكثر مزدحماً للغاية.  
و كما سيستمر بروتوكول TCP في إعادة إرسال القطعة حتى يتم تأكيد استلامها من قبل الوجهة بغض النظر عن الوقت الذي يستغرقه هذا التسليم الموثوق.  
و تطبيقات الوقت الفعلي (real-time applications) تتطلب غالباً حداً أدنى لمعدل الإرسال ولا ترغب في تأخير إرسال القطع بشكل كبير ويمكنها تحمل بعض الفقد في البيانات (tolerate some data loss) فإن نموذج خدمة TCP لا يتناسب بشكل جيد تماماً مع احتياجات هذه التطبيقات.

### 2 - عدم الحاجة لإنشاء اتصال (No connection establishment)

لا يتسبب بروتوكول UDP في حدوث أي تأخير لإرساء اتصال مسبق.  
ولهذا السبب تم تشغيل نظام أسماء النطاقات (DNS) عبر بروتوكول UDP بدلاً من TCP إذ سيكون نظام DNS أبطأ بكثير لو أنه عمل عبر بروتوكول TCP وعلى العكس يستخدم بروتوكول HTTP بروتوكول TCP بدلاً من UDP لأن الموثوقية (reliability) أمر بالغ الأهمية لصفحات الويب التي تحتوي على نصوص.

### 3 - عدم الاحتفاظ بحالة الاتصال (No connection state)

يحافظ بروتوكول TCP على حالة الاتصال في الأنظمة الطرفية (end systems) وتشمل حالة الاتصال هذه: مخازن الاستقبال والإرسال المؤقتة (buffers)، ومعاملات التحكم في الازدحام (congestion-control parameters)، ومعاملات أرقام المتابع والتأكيد (sequence and acknowledgment number parameters).  
أما بروتوكول UDP من ناحية أخرى فلا يحافظ على حالة الاتصال ولا يتتبع أيّاً من هذه المعاملات. ولهذا السبب يمكن للخادم المخصص لتطبيق معين أن يدعم عادةً عدداً أكبر بكثير من العملاء النشطين (active clients) عندما يشغل هذا التطبيق عبر UDP بدلاً من TCP.

### 4 - صغر حجم الترويسة المضافة للحزمة (Small packet header overhead)

تمتلك قطعة بروتوكول TCP (TCP segment) ترويسة زائدة بحجم 20 بايت (20 bytes of header overhead) في كل قطعة بينما بروتوكول UDP يمتلك ترويسة زائدة بحجم 8 بايت فقط (8 bytes of overhead).

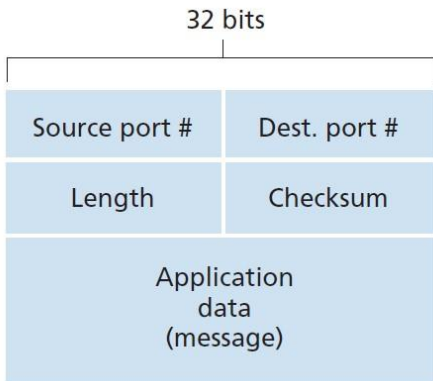
## تطبيقات الإنترنت الشائعة وبروتوكولات طبقة النقل الخاصة بها

(Popular Internet applications and their underlying transport protocols)

| التطبيق (Application)                                               | بروتوكول طبقة التطبيق (Application-Layer Protocol) | بروتوكول طبقة النقل (Transport Layer Protocol) | التطبيق الأساسي (Underlying) |
|---------------------------------------------------------------------|----------------------------------------------------|------------------------------------------------|------------------------------|
| البريد الإلكتروني (Electronic mail)                                 | SMTP                                               | TCP                                            |                              |
| الوصول عن بعد للأجهزة الطرفية (Remote terminal access)              | Telnet                                             | TCP                                            |                              |
| الوصول الآمن عن بعد للأجهزة الطرفية (Secure remote terminal access) | SSH                                                | TCP                                            |                              |
| الويب (Web)                                                         | HTTP, HTTP/3                                       | TCP (لبروتوكول HTTP)، و UDP (لبروتوكول HTTP/3) |                              |
| نقل الملفات (File transfer)                                         | FTP                                                | TCP                                            |                              |
| خادم الملفات عن بعد (Remote file server)                            | NFS                                                | غالباً UDP (Typically UDP)                     |                              |
| البث المتعدد الوسائط (Streaming multimedia)                         | DASH                                               | TCP                                            |                              |
| هواتف الإنترنت / الاتصال الصوتي (Internet telephony)                | بروتوكولات احتكارية عادةً (typically proprietary)  | TCP أو UDP                                     |                              |
| إدارة الشبكات (Network management)                                  | SNMP                                               | غالباً UDP (Typically UDP)                     |                              |
| ترجمة الأسماء / العناوين (Name translation)                         | DNS                                                | غالباً UDP (Typically UDP)                     |                              |



## بنية قطعة بروتوكول UDP (UDP Segment Structure)



- توجد بيانات التطبيق (application data) في حقل البيانات (data field)
- على سبيل المثال في نظام DNS يحتوي حقل البيانات إما على رسالة استعلام (query message) أو رسالة استجابة (response message) أما بالنسبة لتطبيقات بث الصوت (streaming audio) فإن العينات الصوتية هي التي تملأ حقل البيانات.

- تحتوي ترويسة UDP (UDP header) على أربعة حقول فقط يتكون كل منها من بايتين (two bytes) أي 16 بت لكل حقل.
- تسمح أرقام المنافذ (port numbers) للجهاز المضيف المستقبل بمعرفة العملية الصحيحة و ذلك لتمرير بيانات التطبيق إلى العملية المطلوبى أي تنفيذ وظيفة فك التجميع
- يحدد حقل الطول (length field) عدد البايتات في قطعة UDP بأكملها (الترويسة بالإضافة إلى البيانات). وهناك حاجة لوجود قيمة صريحة للطول نظراً لأن حجم حقل البيانات قد يختلف من قطعة UDP إلى أخرى.
- يستخدم المجموع التدقيقي (checksum) من قبل الجهاز المضيف المستقبل للتحقق مما إذا كانت هناك أخطاء قد حدثت أو أُدخلت على القطعة أثناء النقل.

## المجموع التدقيقي لبروتوكول UDP (UDP Checksum)

- يوفر المجموع التدقيقي لبروتوكول UDP (UDP checksum) آلية لاكتشاف الأخطاء حيث أن المجموع التدقيقي يُستخدم لتحديد ما إذا كانت هناك بتات (bits) داخل قطعة UDP قد تغيرت (على سبيل المثال، بسبب التشويش [noise] في روابط الشبكة أو أثناء تخزين القطعة داخل الراوتر) أثناء انتقالها من المصدر إلى الوجهة.
- يقوم بروتوكول UDP عند جهة المرسل (sender side) بحساب متمم الأحاد (1's complement) لمجموع كافة الكلمات بحجم 16 بت (16-bit words) الموجودة في القطعة مع تدوير وإضافة أي زائد (overflow) ينتج أثناء عملية الجمع. يتم وضع هذه النتيجة النهائية في حقل المجموع التدقيقي (checksum field) الخاص بقطعة UDP.

### مثال لحساب Checksum

يُقسَّم المحتوى إلى كلمات من 16 بت كما يلي:

الكلمة الأولى: 0110011001100000

الكلمة الثانية: 0101010101010101

الكلمة الثالثة: 1000111100001100

أولاً - نجمع أول كلمتين جمعاً ثنائياً (تذكر:  $10 = 1+1$  و  $11 = 1+1+1$ ):

$$1011101110110101 = 0101010101010101 + 0110011001100000$$

ثانياً - نجمع الناتج مع الكلمة الثالثة:

$$1011101110110101 + 1000111100001100 = 0100101011000001 \quad 1 \leftarrow \text{هذا الـ 1 على اليسار هو overflow}$$

ثالثاً - نأخذ البت الزائد (Overflow) الناتج من أقصى اليسار ونقوم بتدويره وجمعه مع البت الأقل أهمية في أقصى اليمين

$$0100101011000010 = 1 + 0100101011000001$$

رابعاً - نحسب المتمم الأحادي للناتج (نقلب كل 0 إلى 1 وكل 1 إلى 0):

$$0100101011000010 <----- 10111010100111101$$

يقوم المرسل بوضع هذا الرقم في حقل الـ Checksum داخل ترويسة الـ UDP ويرسل الحزمة.

## التحقق عند المستقبل

يقوم بجمع كل الكلمات المستقبلية (الكلمات الثلاث الأصلية + قيمة الـ Checksum التي حسبها المرسل).

إذا كانت البيانات سليمة (لم يحدث خطأ): يجب أن يكون ناتج الجمع عند المستقبل عبارة عن صف كامل من العدد 1:

111111111111111

إذا ظهر أي صفر (0) في الناتج: يعرف المستقبل فوراً أن هناك بت واحد على الأقل تغير في الطريق بسبب التشويش وبالتالي يعتبر الحزمة تالفة.

**السؤال:** لماذا يوفر بروتوكول UDP مجموع تدقيق (checksum) رغم أن العديد من بروتوكولات طبقة وصل البيانات (link-layer) (بما في ذلك بروتوكول إيثرنت (Ethernet) الشهير) توفر أيضاً ميزة فحص الأخطاء؟

السبب في ذلك هو أنه لا يوجد ضمان بأن جميع الوصلات بين المصدر والوجهة توفر فحصاً للأخطاء أي أن إحدى هذه الوصلات قد تستخدم بروتوكولاً في طبقة وصل البيانات لا يدعم فحص الأخطاء. إضافة إلى ذلك حتى لو تم نقل أجزاء البيانات (segments) بشكل صحيح عبر الوصلة فمن المحتمل أن تحدث أخطاء في البتات (bit errors) عندما يتم تخزين جزء البيانات في ذاكرة جهاز التوجيه (الراوتر).

و بما أن الموثوقية بين الوصلات (link-by-link) واكتشاف الأخطاء داخل الذاكرة ليسا مضمونين، يجب على UDP توفر ميزة اكتشاف الأخطاء في طبقة النقل (transport layer) على أساس "من الطرف إلى الطرف" (end-to-end) وذلك إذا كانت خدمة نقل البيانات بين الطرفين تهدف إلى توفير ميزة اكتشاف الأخطاء.

و من المهم ان نتذكر انه على الرغم من أن UDP يوفر فحصاً للأخطاء إلا أنه لا يفعل أي شيء للتعافي من الخطأ (أو تصحيحه). حيث ان بعض تطبيقات بروتوكول UDP تقوم ببساطة بإهمال (حذف) جزء البيانات المتضرر، بينما يقوم بعضها الآخر بتمرير جزء البيانات المتضرر إلى التطبيق مع إرفاق تحذير.

## النقل الموجه بالاتصال: بروتوكول TCP (Connection-Oriented Transport: TCP)

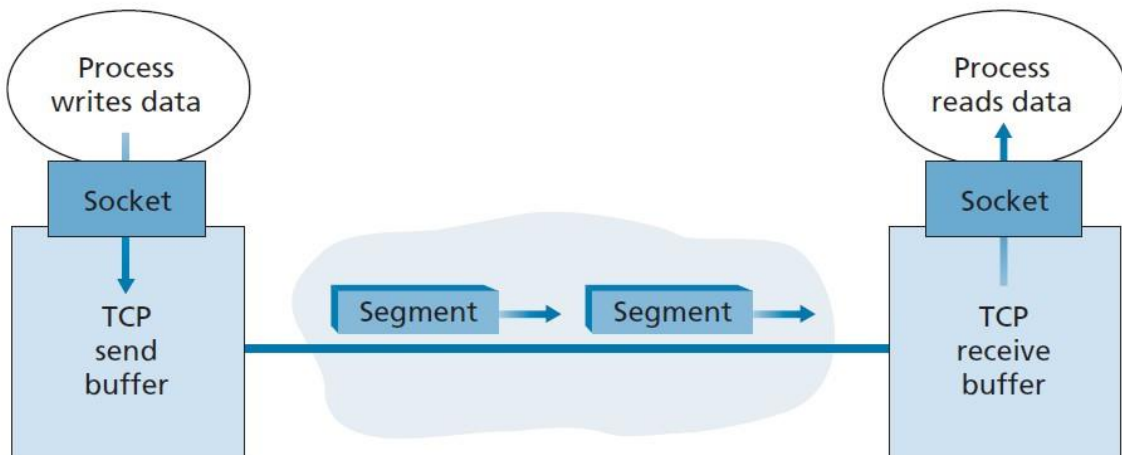
- إن بروتوكول TCP موجه بالاتصال (connection-oriented) لأنه قبل أن تتمكن إحدى العمليات البرمجية للتطبيق (application process) من البدء في إرسال البيانات إلى عملية أخرى يجب على العمليتين أولاً المصافحة (handshake) مع بعضهما البعض أي أنه يجب عليهما إرسال بعض أجزاء البيانات الأولية (preliminary segments) لإنشاء وتحديد معايير (parameters) نقل البيانات اللاحق.
- إن "الاتصال" في بروتوكول TCP ليس عبارة عن دائرة اتصال مادية من طرف إلى طرف تعتمد على تقسيم الوقت (TDM) أو تقسيم التردد (FDM) كما هو الحال في الشبكات المبدلة بالدوائر (circuit-switched networks) أي مثل خطوط الهاتف الأرضي القديمة بدلاً من ذلك فإن هذا "الاتصال" هو اتصال منطقي (logical connection) حيث توجد الحالة المشتركة (common state) فقط داخل بروتوكول TCP في النظامين الطرفين المتواصلين.
- يوفر اتصال TCP خدمة ثنائية الاتجاه بالكامل (full-duplex service) أي الاتصال عبر TCP يسمح بنقل البيانات في الاتجاهين في نفس الوقت بدقة وكفاءة
- يكون اتصال TCP دائماً من نقطة إلى نقطة (point-to-point) أي بين مرسل واحد ومستقبل واحد فقط. لذلك فإن ما يسمى بـ "البث المتعدد" (multicasting) وهو نقل البيانات من مرسل واحد إلى مستقبلين متعددين في عملية إرسال واحدة هي غير ممكن باستخدام بروتوكول TCP

**FINISH WHAT YOU STARTED**

## إنشاء اتصال (TCP Connection Establishment)

## مرحلة الإرسال

- تُسمى العملية البرمجية التي تبدأ في إنشاء الاتصال بـ عملية العميل (client process) بينما تُسمى العملية البرمجية الأخرى بـ عملية الخادم (server process).
- تقوم عملية تطبيق العميل أولاً بإبلاغ طبقة النقل (transport layer) لدى العميل بأنها تريد إنشاء اتصال مع عملية برمجية موجودة في الخادم.
- يرسل العميل أولاً جزء بيانات TCP خاصاً (special TCP segment) ثم يرد الخادم بجزء بيانات TCP خاصٍ ثانٍ وأخيراً يرد العميل مجدداً بجزء بيانات خاصٍ ثالث.
- لا يحمل أول جزء بيانات أي حمولة (payload) أي أنهما لا يحتويان على أي بيانات من طبقة التطبيق بينما قد يحمل الجزء الثالث من هذه الأجزاء حمولة.
- وبسبب إرسال ثلاثة أجزاء بيانات (segments) بين المضيفين فإن إجراء إنشاء الاتصال هذا غالباً ما يُشار إليه باسم المصافحة الثلاثية (three-way handshake).
- بمجرد إنشاء اتصال TCP يمكن لعمليتي التطبيق (application processes) إرسال البيانات إلى بعضهما البعض.
- تمرر عملية العميل (client process) تدفقاً من البيانات عبر المقبس (socket) (الذي يمثل باب العملية البرمجية). وبمجرد أن تمر البيانات عبر هذا الباب تصبح البيانات في عهدة بروتوكول TCP الذي يعمل في جهاز العميل.
- يوجه بروتوكول TCP هذه البيانات إلى مخزن الإرسال المؤقت (send buffer) الخاص بالاتصال وهو أحد المخازن المؤقتة التي يتم تخصيصها وحجزها أثناء عملية المصافحة الثلاثية الأولية (initial three-way handshake).
- من وقت لآخر يقوم بروتوكول TCP بسحب أجزاء من البيانات (chunks of data) من مخزن الإرسال المؤقت ويمررها إلى طبقة الشبكة (network layer)



**ملاحظة :**

إن الحد الأقصى لكمية البيانات التي يمكن سحبها ووضعها في جزء بيانات واحد (segment) تكون محدودة بـ الحجم الأقصى لجزء البيانات (MSS - Maximum Segment Size)

**كيف نحسب ال MSS :**

يتم تحديد قيمة MSS عادةً عن طريق معرفة:

1. معرفة طول أكبر إطار في طبقة وصل البيانات (link-layer frame) يمكن إرساله بواسطة المرسل (وهو ما يُعرف بـ وحدة الإرسال الكبرى (MTU - Maximum Transmission Unit)).
2. خصم وزن (Headers): أي حزمة بيانات تحتاج إلى ترويسة توجيه (ترويسة TCP وترويسة IP) ويبلغ حجمهما معاً في الحالات العادية 40 بايت.
3. نقوم بطرح حجم الترويسة من السعة الكلية للشبكة الناتج هو ال MSS

**مرحلة الاستقبال**

عندما يستقبل بروتوكول TCP جزء بيانات (segment) عند الطرف الآخر يتم وضع بيانات هذا الجزء في مخزن الاستقبال المؤقت (receive buffer) الخاص باتصال TCP. بعد ذلك يقرأ التطبيق تدفق البيانات من هذا المخزن المؤقت.

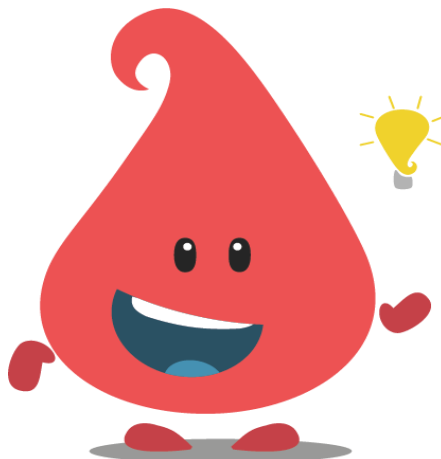
**ملاحظات:**

يتكون اتصال TCP من مخازن مؤقتة (buffers) ومتغيرات (variables) واتصال مقبس بعملية برمجية في أحد المضيفين (hosts) ومجموعة أخرى من المخازن المؤقتة والمتغيرات واتصال مقبس بعملية برمجية في المضيف الآخر.

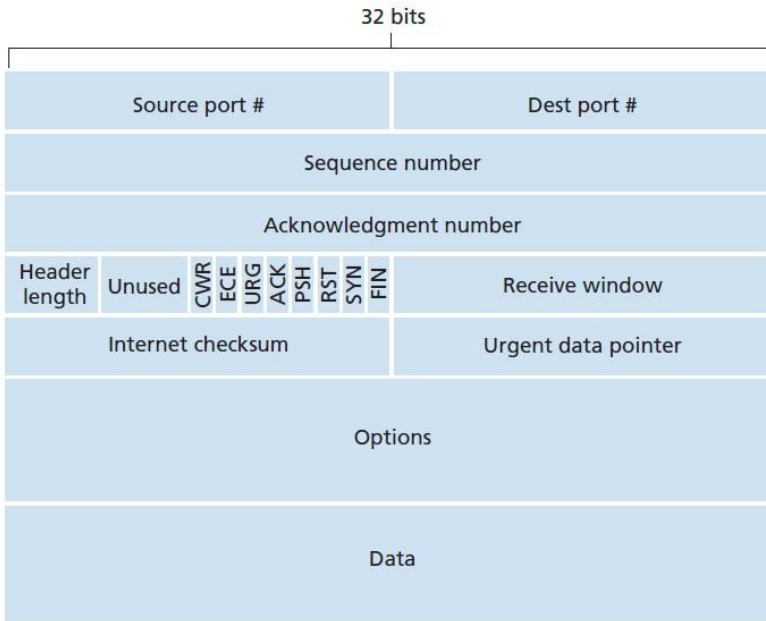
لا يتم تخصيص أي مخازن مؤقتة أو متغيرات للاتصال داخل عناصر الشبكة (مثل أجهزة التوجيه Routers، والمبدلات Switches، والمكررات Repeaters).

**بمعنى :** أن جميع أجهزة الشبكة ك الراوتر و غيره لا يخصصوا أي مخزن للاحتفاظ بأي بيانات عن عملية الاتصال

التخزين يحدث فقط عند المرسل و المستقبل .



## بنية جزء بيانات (TCP Segment Structure)



- يتكون جزء بيانات TCP من حقول الترويسة (header fields) وحقل البيانات (data field)
- تتضمن الترويسة أرقام منافذ المصدر والوجهة (source and destination port numbers) والتي تُستخدم لتجميع/تفكيك (multiplexing/demultiplexing) البيانات من وإلى تطبيقات الطبقات العليا.
- تتضمن الترويسة حقل مجموع التدقيق (checksum field)

- يستخدم حقل رقم التسلسل المكون من 32 بت (32-bit sequence number field) وحقل رقم التأكيد المكون من 32 بت (32-bit acknowledgment number field) من قبل مرسل ومستقبل TCP لتطبيق خدمة نقل بيانات موثوقة. (reliable data transfer service)

- و يستخدم حقل نافذة الاستقبال المكون من 16 بت (16-bit receive window field) من أجل التحكم في التدفق (flow control) للإشارة إلى عدد البايتات التي يستطيع المستقبل قبولها.
- حقل طول الترويسة المكون من 4 بتات (4-bit header length field) وظيفته إخبار نظام التشغيل أين تنتهي ترويسة الـ TCP وتبدأ البيانات الحقيقية و يمكن لترويسة TCP أن تكون ذات طول متغير بسبب حقل الخيارات (TCP options field) عادةً ما يكون حقل الخيارات فارغاً بحيث يبلغ طول ترويسة TCP النموذجية 20 بايت .

### ملاحظة : في حقل الترويسة :

وحدة القياس ليست بالبايت بل بالكلمات (Words) حيث كل وحدة داخل هذا الحقل تساوي "كلمة من 32 بت" (وهي تعادل 4 بايت تماماً) .

أي في الحالة النموذجية (عندما لا تستخدم الحزمة حقل الخيارات (Options) يكون حجم الترويسة الثابت هو 20 بايت) يكون :  $20 / 4 = 5$

أي سيكون حقل طول الترويسة 5

يحتوي حقل الأعلام (flag field) على 6 بتات:

- يُستخدم بت التأكيد (ACK bit) للإشارة إلى أن القيمة المحمولة في حقل رقم التأكيد (acknowledgment number field) هي قيمة صالحة وهذا يعني أن جزء البيانات (segment) يحتوي على إشعار باستلام جزء بيانات آخر تم استقبله بنجاح.
- تُستخدم البتات RST، و SYN، و FIN من أجل إنشاء الاتصال (connection setup) وإنهاء الاتصال (teardown).
- يُستخدم البتان CWR و ECE من أجل الإشعار الصريح بالازدحام (explicit congestion notification) داخل الشبكة.
- يشير ضبط بت الدفع (PSH bit) إلى أنه يجب على المستقبل تمرير البيانات إلى الطبقة العليا (application layer) فوراً دون تأخير.
- يُستخدم بت البيانات العاجلة (URG bit) للإشارة إلى وجود بيانات داخل هذا الجزء (segment) تم تمييزها بواسطة الجهة المرسلة في الطبقة العليا بأنها "عاجلة". ويتم تحديد موقع البايت الأخير من هذه البيانات العاجلة بواسطة حقل مؤشر البيانات العاجلة المكون من 16 بت. (16-bit urgent data pointer field). حيث يجب على بروتوكول TCP إعلام الكيان البرمجي في الطبقة العليا لدى الطرف المستقبل عند وجود بيانات عاجلة، وتمرير مؤشر يشير إلى نهاية هذه البيانات العاجلة.

فائدة أريسية :

لنوضح عمل حقل الخيارات options field :

الوضع الافتراضي لترويسة TCP هو 20 بايت ولكن إذا أراد الطرفان (المرسل والمستقبل) ميزات إضافية متطورة، يتم تفعيل حقل "الخيارات" ليمتد حجم الترويسة حتى 60 بايت كحد أقصى. بحيث يستخدم حقل الخيارات :

1. التفاوض على الحجم الأقصى للحزمة (MSS Negotiation)

- الفكرة: كما ذكرنا سابقاً الـ MSS هو الحجم الصافي للبيانات داخل حزمة الـ TCP
- كيف يُستخدم حقل الخيارات هنا؟ أثناء عملية المصافحة الثلاثية (Three-Way Handshake)، يضع العميل في حقل الخيارات قيمة الـ MSS التي يفضلها (مثلاً 1460 بايت). ويرد السيرفر في حقل الخيارات الخاص به بالقيمة التي يفضلها هو أيضاً.
- النتيجة: يتفق الطرفان على الحجم الأقل بينهما لاعتماده طوال فترة الاتصال لضمان عدم تقطيع الحزم (Fragmentation).

2. عامل قياس النافذة (Window Scaling Factor)

■ **المشكلة في الشبكات الحديثة:** يحتوي بروتوكول TCP على حقل أساسي اسمه نافذة الاستقبال (Receive Window) وحجمه الثابت هو 16 بت فقط. هذا يعني أن أقصى كمية بيانات يمكن للمستقبل أن يطلبها دفعة واحدة هي  $16^2$  (حوالي 64 كيلوبايت). في شبكات الألياف الضوئية الحديثة عالية السرعة (High-Speed Networks)، هذا الرقم يعتبر صغيراً جداً ويجعل الشبكة بطيئة لأن المرسل يضطر للتوقف والانتظار كثيراً.

**الحل عبر حقل الخيارات:** يتيح خيار الـ Window Scaling إضافة معامل ضرب (Multiplier) هذا المعامل يقوم بعملية إزاحة بتات (Bit Shift) ترفع حجم النافذة بشكل افتراضي من 64 كيلوبايت لتصل إلى 1 جيجابايت

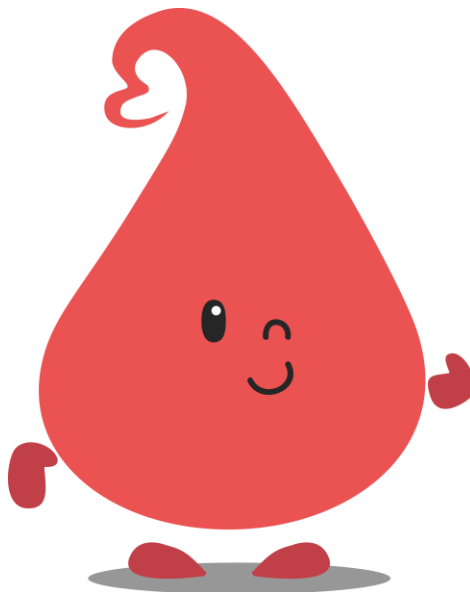
**النتيجة:** يسمح للشبكات السريعة بنقل كميات هائلة من البيانات دون توقف، وهو أمر أساسي في الإنترنت اليوم.

#### الطابع الزمني (Time-Stamping)

الفكرة: يحتاج TCP لحساب الوقت الذي تستغرقه الحزمة للذهاب والعودة، ما يُعرف بـ Round Trip Time (RTT)، لكي يعرف متى يقرر أن الحزمة ضاعت ويجب إعادة إرسالها.

**كيف يعمل؟** يضع المرسل في حقل الخيارات "طابعاً زمنياً" (مثلاً: أرسلت الساعة 10:00:00.001). وعندما يستقبلها الطرف الآخر ويرد بإشعار الاستلام (ACK) يعيد إرفاق نفس الطابع الزمني.

**النتيجة:** يقرأ المرسل الطابع الزمني فور وصول الـ ACK ويطرحه من الوقت الحالي فيحسب الـ RTT بدقة متناهية بالملي ثانية، مما يحسن من كفاءة معالجة الحزم الضائعة.



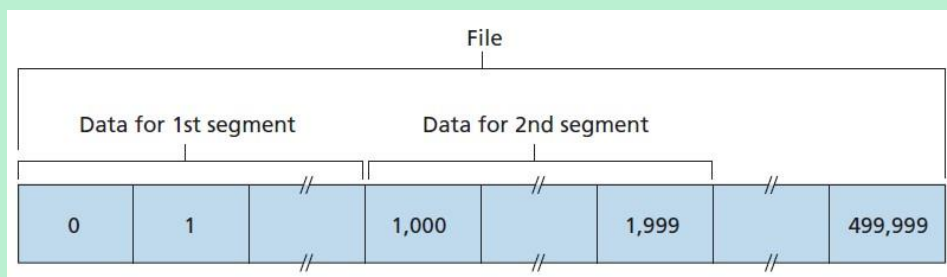
## أرقام التسلسل وأرقام التأكيد

## (Sequence Numbers and Acknowledgment Numbers)

رقم التسلسل (sequence number) الخاص بجزء البيانات (segment) هو رقم تدفق البايتات (byte-stream number) لأول بايت موجود داخل جزء البيانات هذا (أي هو ترتيب أول بايت (Byte) تحمله هذه الحزمة داخل الملف الكلي).

مثال:

حجم الملف الكلي 500,000 بايت.



الحجم الأقصى للحزمة الواحدة 1,000: (MSS) بايت (أي أن كل حزمة تستطيع حمل 1,000 بايت فقط كحد أقصى من البيانات الصافية).

▪ **ترقيم البايتات:** يبدأ الكمبيوتر بترقيم بايتات الملف من الرقم (البايت الأول رقم 0، الثاني رقم 1، الثالث رقم 2... حتى البايت الأخير رقم 499,999).

لإرسال هذا الملف، سيقوم بروتوكول TCP بتقسيمه إلى 500 حزمة  $500,000 \div 1,000 = 500$ . كيف سيضع أرقام التسلسل (Sequence Numbers) لها؟

**ترتيب الحزم الرقمي:**

▪ الحزمة الأولى : (Segment 1)

تحمل البايتات من الرقم 0 إلى الرقم 999.

رقم التسلسل لها Sequence Number = 0 لأن أول بايت داخلها رقمه 0.

▪ الحزمة الثانية : (Segment 2)

تحمل البايتات من الرقم 1000 إلى الرقم 1999.

رقم التسلسل لها Sequence Number = 1000 لأن أول بايت داخلها رقمه 1000.

▪ الحزمة الثالثة : (Segment 3)

تحمل البايتات من الرقم 2000 إلى الرقم 2999

رقم التسلسل لها Sequence Number = 2000 لأن أول بايت داخلها رقمه 2000

- بروتوكول TCP ثنائي الاتجاه بالكامل (Full-Duplex)، ولذلك يمكن للمضيف أ (Host A) استقبال البيانات من المضيف ب (Host B) في نفس الوقت الذي يرسل فيه بيانات إلى المضيف ب (كجزء من اتصال TCP نفسه).
- كل جزء بيانات (segment) يصل من المضيف ب يحتوي على رقم تسلسل (sequence number) خاص بالبيانات المتدفقة من ب إلى أ.
- رقم التأكيد (acknowledgment number) الذي يضعه المضيف أ في جزء البيانات الخاص به هو رقم البايت التالي الذي يتوقع المضيف أ استقبله من المضيف ب.

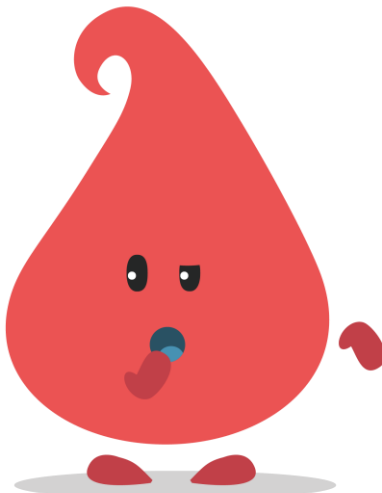
## مثال 1

استقبل المضيف أ جميع البايتات ذات الأرقام من 0 إلى 535 من المضيف ب، ونفترض أنه على وشك إرسال جزء بيانات (حزمة) إلى المضيف ب. ينتظر المضيف أ الآن البايت رقم 536 وجميع البايتات اللاحقة في تدفق بيانات المضيف ب. لذلك، يضع المضيف أ الرقم 536 في حقل رقم التأكيد (acknowledgment number field) ضمن جزء البيانات الذي يرسله إلى ب.

## مثال 2

استقبل المضيف أ (Host A) جزء بيانات واحد (segment) من المضيف ب (Host B) يحتوي على البايتات من 0 إلى 535، وجزء بيانات آخر يحتوي على البايتات من 900 إلى 1,000. لسبب ما، لم يستقبل المضيف أ البايتات الممتدة من 536 إلى 899 حتى الآن. ما يزال المضيف أ ينتظر البايت رقم 536 (وما بعده) لكي يتمكن من إعادة بناء تدفق بيانات المضيف ب. بناءً على ذلك، فإن جزء البيانات التالي الذي سيرسله أ إلى ب سيحتوي على الرقم 536 في حقل رقم التأكيد (acknowledgment number field).

**نتيجة هامة :** لأن بروتوكول TCP يؤكد فقط على استلام البايتات المتتالية وصولاً إلى أول بايت مفقود في التدفق، يُقال إن بروتوكول TCP يقدم **تأكيدات تراكمية (cumulative acknowledgments)**.



السؤال: ماذا يفعل المضيف عندما يستقبل أجزاء بيانات وصلت غير مرتبة (out-of-order segments) في اتصال TCP ؟

هناك خياران متاحان لتطبيق بروتوكول TCP في هذه الحالة:

إما (1): أن يقوم المستقبل بإهمال/رمي أجزاء البيانات غير المرتبة فوراً (وهو خيار يمكن أن يُبسّط تصميم آلية الاستقبال في نظام التشغيل)

أو (2): أن يقوم المستقبل بالاحتفاظ بالبايتات غير المرتبة و ينتظر البايتات المفقودة لكي تملأ الفجوات الحاصلة. هذا الخيار يعتبر أكثر كفاءة من حيث استغلال عرض نطاق الشبكة (bandwidth)، وهو النهج المتبع والمعتمد في الممارسة العملية (in practice).

ملاحظة:

يختار كلا طرفي اتصال TCP رقم تسلسل بدائي بشكل عشوائي (randomly choose an initial sequence number) (وليس 0 مثل المثال السابق ولكن استخدمناه في المثال للوضوح).

و لكن لماذا يفضل توليد رقم عشوائي ؟

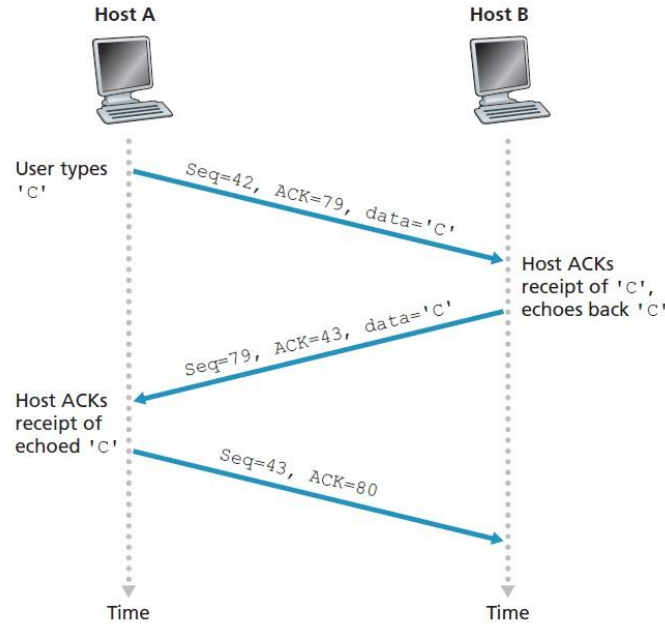
يتم القيام بذلك لتقليل احتمالية أن يتم الالتباس واعتبار جزء بيانات (segment) ما يزال موجوداً وتائهاً في الشبكة من اتصال سابق تم إنهاؤه بالفعل بين المضيفين) على أنه جزء بيانات صالح وصحيح في اتصال لاحق بين هذين المضيفين نفسهما (واللاذين قد يصادف أنهما يستخدمان نفس أرقام المنافذ "port numbers" التي كانت مستخدمة في الاتصال القديم).

## تطبيق Telnet عبر TCP - أرقام التسلسل والتأكيد (Sequence and Acknowledgment Numbers)

إن بروتوكول Telnet هو بروتوكول شهير في طبقة التطبيقات (application-layer) يُستخدم لتسجيل الدخول عن بُعد (remote login) ويعد Telnet تطبيقاً تفاعلياً (interactive application). كل محرف (character) يقوم المستخدم بكتابته عند جهة العميل (Client) سيتم إرساله إلى المضيف البعيد. يقوم المضيف البعيد بإرسال نسخة بدوره من كل محرف واستعادتها (echo back) ليتم عرضها على شاشة مستخدم Telnet. تستخدم آلية «إعادة الصدى» هذه لضمان أن المحارف التي يراها المستخدم قد تم استقبالها ومعالجتها بالفعل في الموقع البعيد.

ملاحظة

Piggybacking: تسمى عملية دمج إشعار التأكيد (ACK) مع بيانات حقيقية صادرة في نفس الحزمة باسم Piggybacking وهي توفر استهلاك الشبكة.



## الخطوة 1: العميل (Host A) يرسل حرفاً إلى السيرفر (Host B)

ماذا حدث؟ قام المستخدم بكتابة الحرف 'C'.

بيانات الحزمة:

- Seq = 42: هذا هو رقم التسلسل الحالي للبايت الذي يرسله العميل.
- ACK = 79: العميل يخبر السيرفر: "أنا بانتظار البايت رقم 79 منك".
- Data = 'C': الحمولة هي الحرف نفسه وحجمه (1 بايت).

## الخطوة 2: السيرفر (Host B) يرد بالـ ACK ويرسل صدى الحرف (Echo)

ماذا حدث؟ استلم السيرفر الحرف بنجاح، والآن يود تأكيد الاستلام (ACK) وإعادة إرسال الحرف للشاشة (Data = 'C') في حزمة واحدة.

بيانات الحزمة:

- Seq = 79: التزم السيرفر برغبة العميل بما أن العميل طلب البايت 79 فقد خرجت الحزمة برقم تسلسل 79.
- ACK = 43: كيف حسب السيرفر هذا الرقم؟ الحزمة السابقة من العميل بدأت بـ Seq=42 وحملت بايت واحد فقط السيرفر يقول للعميل: "وصلني البايت 42، وأنا الآن أنتظر منك البايت رقم 43".
- Data = 'C': إعادة الحرف مجدداً للعميل ليظهر على شاشته.

### الخطوة 3: العميل (Host A) يؤكد استلام صدى الحرف

ماذا حدث؟ استلم العميل الحرف وظهر على شاشته، والآن يرسل حزمة أخيرة للسيرفر ليؤكد له أنه استلم الصدى بنجاح.

بيانات الحزمة:

- Seq = 43: العميل التزم بالرقم الذي طلبه السيرفر في الخطوة السابقة.
- ACK = 80: السيرفر في الخطوة السابقة أرسل بايت واحد Seq=79
- حقل البيانات فارغ: العميل هنا لا يملك أي حرف جديد ليرسله الحزمة وظيفتها فقط التأكيد لذا لم يحدث هنا تحميل (No Piggybacking).

## انتهت المحاضرة

